Organización de computadores
Taller - Lenguaje ensamblador y traducción de instrucciones

MBA, I.S. José Luis Montoya Pareja
Departamento de Informática y Sistemas
Universidad EAFIT
Medellín, Colombia, Suramérica

Solucionado por: Isabella Ocampo Sánchez

Objetivo

Aplicar los conocimientos de clase para:

1. Implementar comandos propuestos en el enunciado con los comandos tipo A y C del lenguaje ensamblador del Nand2Tetris
2. Traducir programas cortos de Ensamblador a Binario
3. Traducir programas en binario a ensablador (proceso de disassembly)

Ejercicios

Suponga que tiene la siguiente tabla con los siguientes valores:

Es simplemente una etiqueta

Nombre	Dirección
j	3012
sum	4500
q	3812
arr	20561
sum	2200
x	4000
i	6151
END	20
NEXT	16

Tabla 1. Tabla de direcciones

Asuma que los valores que aparecen en el campo **Dirección** son las direcciones en memoria RAM donde están creadas las variables descritas en el campo **Nombre**.

1. Implemente en assembler Hack las siguientes tareas:

```
a. sum = 0
b. j = j + 1
c. q = sum + 12 - j
d. arr[3] = -1
e. arr[j] = 0
f. arr[j] = 17
g. goto 50
h. if D==0 goto 112
i. if D<9 goto 507
j. if RAM[12] > 0 goto 50
k. if sum>0 goto END
l. if x[i]<=0 goto NEXT
```

2. Traduzca de lenguaje ensamblador a binario las siguientes instrucciones, indicando paso a paso el proceso:

a. @sum	f. M=D
b. M=0	g. @i
c. @0	h. MD=M-1
d. D=M	i. @NEXT
e. @i	j. D;JLT

PA6

2-3-4

Clasificación Instrucción

3. Investigue en qué consiste realizar el desensamblado de un programa.

Consiste en transformar el binario a instrucciones tipo A y tipo C.

4. Una vez lo tenga claro, realice el desensamble de los siguientes números en binario que se encontraron en una memoria RAM de Hack y determine qué hace el programa:

A	0000000000010000	0000000000010000 = @16
C	1110101010001000	M=0
A	0000000000000000	0000000000000000 @0
C	1111110000010000	D=M
A	0000000000010001	0000000000010001 = @17
C	1110001100001000	M=D
A	0000000000010001	0000000000010001 = @17
C	1111110010011000	DM=M-1
A	0000000000010000	0000000000010000 = @16
C	1110001100000100	D;JLT
A	0000000000000001	0000000000000001 = @1
C	1111110000010000	D=M
A	0000000000010000	0000000000010000 = @16
C	1111000010001000	M=D+M
A	0000000000000110	0000000000000110 = @6
C	1110101010000111	0;JMP
A	0000000000010000	=@16
C	1111110000010000	D=M
A	0000000000000010	0000000000000010 =@2
C	1110001100001000	M=D
A	0000000000010100	0000000000010100 =@20
C	1110101010000111	0;JMP

Traduzca a Tipo A / Tipo C

1. Implemente en assembler Hack las siguientes tareas:

```
a. sum = 0
b. j = j + 1
c. q = sum + 12 - j
d. arr[3] = -1
e. arr[j] = 0
f. arr[j] = 17
g. goto 50
h. if D==0 goto 112
i. if D<9 goto 507
j. if RAM[12] > 0 goto 50
k. if sum>0 goto END
l. if x[i]<=0 goto NEXT
```

1

a) // sum=0 ó sum←0

```
@sum
M=0
```

b) // j=j+1 ó j←j+1

```
@j
M=M+1
```

c) // q=sum+12-j ó q←sum+12-j

```
@sum
D=M
@12
D=D+A
@j
D=D-A
@q
M=D
```

d) arr[3]=-1 , arr[3]←-1

"Cuando hablamos de arr le sumamos [n] a la posición del arreglo".

```
@arr
D=A
@3
D=D+A
A=D
M=-1
```

e) arr[j]=0

```
@arr
D=A
@j
D=D+A
A=D
M=0
```

f) //arr[j]=17

Esto es una variable, así que refiere a lo que hay dentro

```
@arr
D=A
@j
D=D+M
@13
M=D
@17
D=A
@13
A=M
M=D
```

Saltar incondicionalmente

g) // goto 50 @destino -> cargar

```
@50
0;JMP
```

h) if D==0 goto 112

```
@112
D;JEQ
```

D-9<0

i) if D<9 goto 507

```
@9
D=D-A
@507
D;JLT
```

Salto condicional

j) if RAM[12]>0 goto 50

```
@12
D=M
@50
D;JGT
```

k) if sum>0 goto END

```
@sum
D=M
@END
D;JGT
```

l) if x[i]<=0 goto NEXT

variable es un dato

```
@x
D=A
@i
A=D+M
D=M
@NEXT
D;JLE
```

FORMA #1 CON LAS ETIQUETAS

Traduzca a Tipo A / Tipo C

1. Implemente en assembler Hack las siguientes tareas:

a. sum = 0
b. j = j + 1
c. q = sum + 12 - j
d. arr[3] = -1
e. arr[j] = 0
f. arr[j] = 17
g. goto 50
h. if D==0 goto 112
i. if D<9 goto 507
j. if RAM[12] > 0 goto 50
k. if sum>0 goto END
l. if x[i]<=0 goto NEXT

1

a) // sum=0 ó sum←0

```
@4500
M=0
```

b) // j = j + 1 ó j ← j+1

```
@3012
M= M+1
```

c) // q = sum + 12 - j ó
q ← sum + 12 - j

```
@4500
D=M
@12
D=D+A
@3012
D=D-M
@3812
M=D
```

d) arr[3] = -1 , arr[3] ← -1
"Cuando hablamos de arr le sumamos [n] a la posición del arreglo".

```
@3
D=M
@20561
A=D+A
M=-1
```

e) arr[j] = 0

```
@3012
D=M
@20561
A=D+A
M=0
```

f) // arr[j] = 17

Esto es una variable, así que refiere a lo que hay dentro

```
@3012
D=M
@20561
D=D+A
@15
M=D
@17
D=A
@15
A=M
M=D
```

Saltar incondicionalmente

g) // goto 50 @destino → cargar

```
@50
0;JMP
```

h) if D==0 goto 112

```
@112
D;JEQ
```

i) if D<9 goto 507

```
@9
D=D-A
@507
D;JLT
```

Salto condicional

j) if RAM[12] >0 goto 50

```
@12
D=M
@50
D;JGT
```

k) if sum>0 goto END

```
@4500
D=M
@20
D;JGT
```

l) if x[i] <=0 goto NEXT

```
@6151
D=M
@4000
A=D+A
D=M
@16
D;JLE
```

Nombre	Dirección
j	3012
sum	4500
q	3812
arr	20561
sum	2200
x	4000
i	6151
END	20
NEXT	16

Tabla 1. Tabla de direcciones

FORMA #2 PROFE

2. Traduzca de lenguaje ensamblador a binario las siguientes instrucciones, indicando paso a paso el proceso:

a. @sum
b. M=0
c. @0
d. D=M
e. @i
f. M=D
g. @i
h. MD=M-1
i. @NEXT
j. D;JLT

Tipo de Instrucción

Tener claro: (2)

1. Cada instrucción se compone de 16 bits
2. Para tipo C es

31633 ⇒ Nemotecnia

3 ⇒ 111
1 ⇒ a
6 ⇒ cccccc
3 ⇒ ddd
3 → JJJ

3. Para tipo A: 0 _____

valor en binario de la constante

Miro el comp = ☐

Si opero con M
↳ a=0

Opero con #, D, A
↳ a=1

a) @sum ⇒ A

4500 | 2 ⇒ R=0
05 2250
10
00

2250 | 2 ⇒ R=0
02 1125
05
10
0

1125 | 2 ⇒ R=1
12 562
05
1

562 | 2 ⇒ R=0
16 281
02
0

281 | 2 ⇒ R=1
08 140
01

140 | 2 ⇒ R=0
00 70

70 | 2 ⇒ R=0
10 35
0

35 | 2 ⇒ R=1
15 17
1

17 | 2 ⇒ R=1
1 8

8 | 2 ⇒ R=0
0 4

4 | 2 ⇒ R=0
0 2

⇒ 2 | 2 ⇒ R=0
0 1

1 | 2 ⇒ R=1

CONVERSIÓN A BINARIO

0 00 1000110010100

"se diferencia por el @

3

b) M=D acdj 31633

1 1 1 0 1 0 1 0 1 0 0 0 1 0 0 0

c) @0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

d) D=M

1 1 1 1 1 1 0 0 0 0 0 1 0 0 0 0

e) @i

= 0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1

i = 6151

6151 | 2
015 3075 | 2
11   10 1537 | 2
1   07 768 | 2
     15 384 | 2
     1   1 08 18 192 | 2
         0 04 12 96 | 2
            0 0 16 48 | 2
               0 08 24 | 2
                  0 04 12 | 2
                     0 0 6 | 2
                        0 3 | 2
                           1 1

F) M=D 31633

1 1 1 0 0 0 1 1 0 0 0 0 1 0 0 0

g) @i

0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1

h) MD=M-1

1 1 1 1 1 1 0 0 1 0 0 1 1 0 0 0

i) @NEXT

= 0000000000010000

NEXT = 16

16 | 2
0 8 | 2
0 4 | 2
0 2 | 2
0 1

j) D; JLT

1110001100000100